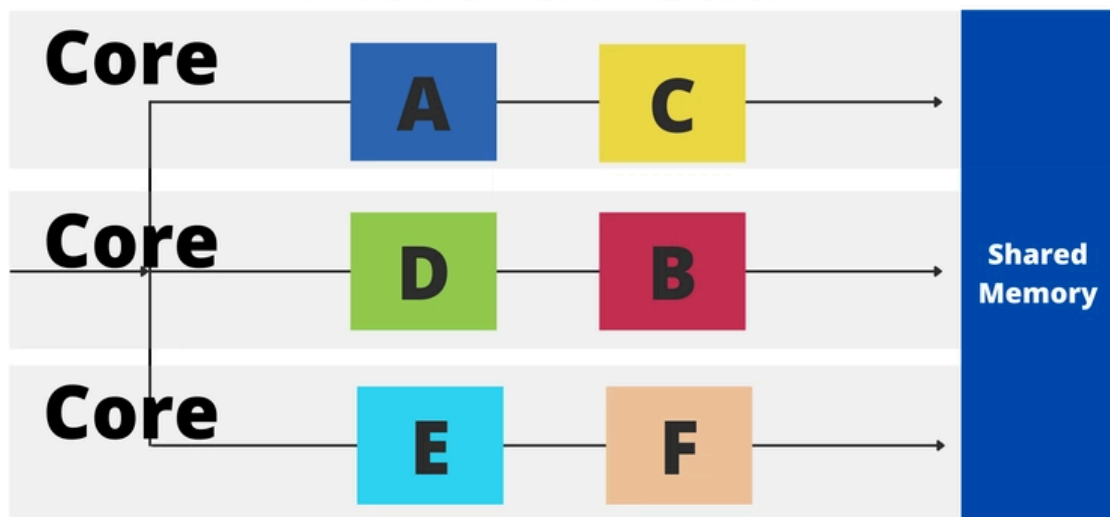


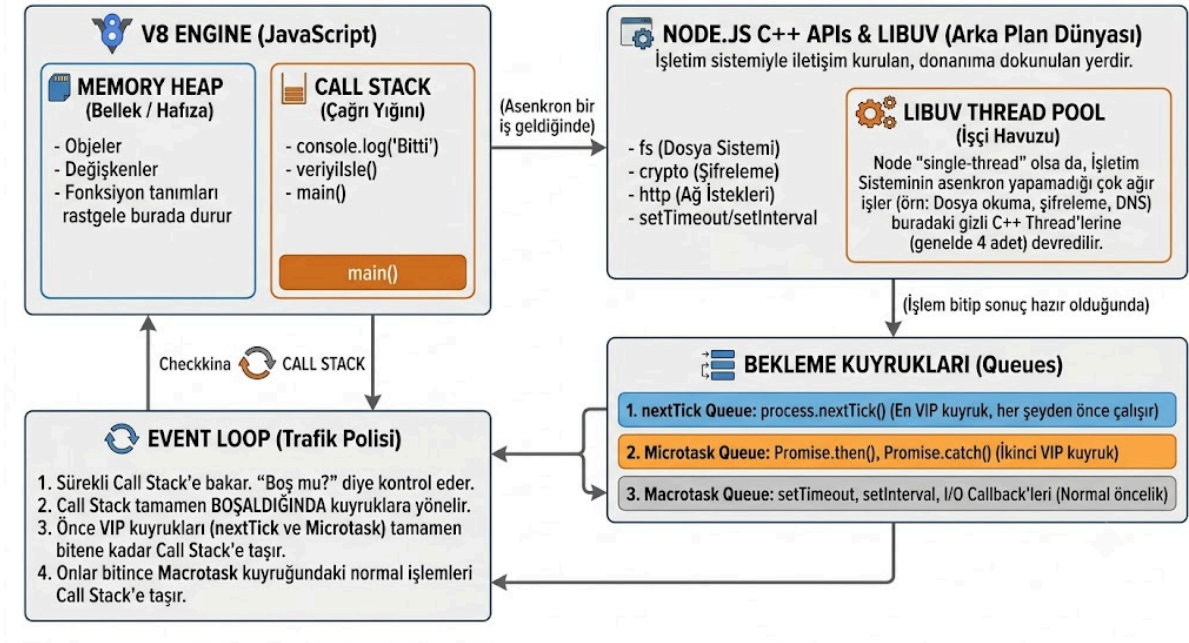
Node js single thread midir multi thread mi?

Single thread



Multi thread





Memory değişkenleri objeleri ve fonksiyonları hafızada random olarak tutar.

Call Stack kaynak programı main den satır satır çalıştırır. OS işlemleri için NodeJS C++ APIs ve LIBUV Thread Pool ile os kesmelerini Javascript CPU Core dan bağımsız bir CPU core da çalıştırır.

Tamamlanan OS kesme işlemleri Queue ya düşer ve önceliklerine göre sıralandırılırlar.

Event Loop sürekli olarak Call Stack i kontrol eder ve boş bulduğunda Queue ya bakar tamamlanan işlem, sorgu varsa getirir.

Worker Threads senkron yapıdadır. Worker Threads te ayrı bir JavaScript motoru ve ayrı bir Event Loop oluşur. Başka bir CPU Core da çalışır.

Ayrıca her C++ Thread i ayrı bir CPU Core da çalışır.

Libuv, aşırı yük getirmeden yaygın bloklayan (blocking) işlemleri (dosya sistemi okuma/yazma, DNS çözünme, kriptografik işlemler olmak üzere default olarak 4 thread oluşturur. Biz thread sayısına manuel müdahalede bulunabiliriz.

Bu manuel thread atama işlemini büyük boyutlu dosyalar için kullanırız(görüntü işleme, kriptografi işlemleri)

Burada dikkat edilmesi gereken nokta donanımdaki CPU Core sayısıdır. Thread sayısını istediğimiz kadar arttırabiliriz ama Core sayımız azsa eğer performans bakımından istenen sonuçları alamayız.

RACE CONDITION

Birden fazla thread in aynı kaynağa ulaşmaya çalışmasıdır. Kaynak manipölasyonundan dolayı ortaya çıkabilecek hataları önlemek adına şu önlemler alınır:

Bir thread veriye erişir durumdayken o veriyi kilitlet ve işlem bitene kadar erişime kapatır.

Semafor Wait and Signal ile çalışır. Wait te clock kontrol edilir. Sayac>0 sa 1 azaltılır ve kaynağa erişmeye devam eder; eğer Sayac=0 sa kaynaklar doluysa thread uyku moduna geçer

Signal da ise kaynağı kullanan thread işlemi tamamlayınca sinyal verir ve Sayac=Sayac+1 olur ve sırada bekleyen thread ler kaynağı kullanmaya başlar

Atomic Operations ise bölünmez ve tamamlanması gereken işlemlerdir. Örn. Sayac=Sayac+1 işlemi başlayınca gerçekleşip tamamlanması gereken bir işlemdir. Bu işlemlerde aynı anda birden fazla thread in erişiminin engellenmesi için kaynak kitlenir.

NodeJS te Race Condition yaşanır nasıl mı?

Call Stack ilk başta increment() i okur ve bu bir db işlemi olduğu için NodeJS C++ Core da çalışması için gönderilir. 2. Bir işlem C++ Core una gönderilirse **race condition** durumu yaşanır.

Aşağıdaki kod örneğinde sadece add metodu mutex ile korunabiliyor. Bu da demek oluyor ki iki kullanıcı aynı kullanıcı profili/hesabı ile read/write balance yapabilir ama add balance yapabilmek için Çözümlenmiş Promise ten başlayan sıraya girmeliler ve sırası gelen işlem add yapabilmeli.

Kullanıcı başına nesne oluşturulur. Yani bir kullanıcı sisteme bir defa giriş yaptıysa 1 instance oluşur başka bir cihazdan 2. Bir girişte aynı instance kullanılır. Bu şekilde race condition önlenir.

Worker lar sadece SharedArrayBuffer ile shared memory erişimi yaparlar.

SharedArrayBuffer ile birden fazla thread in shared memory e ulaşabilmesini sağlarız.

Worker thread te race condition önlemi almadığımız için her görevi ayrı bir thread sıra beklemeden yapmaya çalışır ve race condition yaşanır.

NodeJS OS işlemleri olmadan single thread çalışır ama Worker class instance ları ile multithread olarak çalışabilir.

Child process ile thread farkına gelirsek process ler farklı memory alanlarını kullanır; threads ise shared memory dir aynı memory alanları kullanılır.

Cluster ve non-cluster durummlarını kıyaslarsak non-cluster single thread tir ve oluşan instance lar ve Mutex nesneleri aynı sunucuda bulunur. Eğer birden fazla sunucu varsa bu durumda oluşan Mutex instance lar farklı sunucularda olacak. Bu durum cluster la sağlanır. Her sunucu asıl sunucunun bir kopyasıdır. Bu durumda oluşan aynı profile ait ama farklı sunucularda bulunan Mutex nesnelerinin kontrolünü Sabit bir Redis sunucusu ile sağlarız. Bu anahtarlar nodejs sunucuları yerine Bağımsız Redis sunucularında tutulur ve her Mutex gerektiren işlemde redis sunucusundan o anahtarın kullanımda olup olmadığı kontrol edilir.

Worker ve cluster arasındaki fark clustering bir servise çok fazla istek geliyorsa eğer o servisin kopyalarını ayrı process olarak oluşturur ve load balance sağlar container yönetimine benziyor worker sa eldeki işi multithread yapıda işlememlizi sağlıyor. yani eldeki işi tek thread yerine birden fazla thread e paylaştırıyor

RACE CONDITION HATASI OLAN KOD :

```

const {
  Worker,
  isMainThread,
  workerData,
} = require("worker_threads");

const WORKER_COUNT = 4; // 4 ayrı thread
const INCREMENTS = 200000; // race i görebilmek için her thread 200000
defa +1 yapacak

if (isMainThread) {
  // ANA THREAD
  // Paylaşılan bellek: tüm thread'ler aynı 4 byte'a bakar.
  const sharedBuffer = new
SharedArrayBuffer(Int32Array.BYTES_PER_ELEMENT);
  const view = new Int32Array(sharedBuffer);
  view[0] = 0; // balance = 0

  const workers = [];
  for (let i = 0; i < WORKER_COUNT; i++) {
    // Aynı dosyayı worker olarak yeniden çalıştırıyoruz;
SharedArrayBuffer'ı
    // workerData ile geçiyoruz (kopyalanmaz, AYNI bellek paylaşılır).
    workers.push(
      new Promise((resolve, reject) => {
        const w = new Worker(__filename, {
          workerData: { sharedBuffer, increments: INCREMENTS },
        });
        w.on("error", reject);
        w.on("exit", (code) =>
          code === 0 ? resolve() : reject(new Error("worker exit " +
code)));
      })
    );
  }

  Promise.all(workers).then(() => {
    const expected = WORKER_COUNT * INCREMENTS;
    const actual = view[0];
    console.log("Beklenen bakiye:", expected);
    console.log(

```

```
        "Gerçek bakiye:  ",
        actual,
        actual === expected ? "tesadüf" : "race condition"
    );
    console.log("Kaybolan artış:  ", expected - actual);
});
} else {
    // --- WORKER THREAD ---
    const view = new Int32Array(workerData.sharedBuffer);
    for (let i = 0; i < workerData.increments; i++) {
        // ATOMİK DEĞİL: oku-değiştir-yaz arasında başka thread araya
        girebilir.
        view[0] = view[0] + 1;
    }
}
```